

Natural Language Processing – Assignment 3: Transformers & IR

Tel Aviv University

Student Name(s): Maxim German, Ilay Abramovich, Eram Shufaro, Itay Hazan
ID(s): 322542887, 322271032, 209074731, 209277367

Due: January 25, 2026
Lecturer: Dr. Tal Wagner
TA: Idan Tarshish

Contents

1	Q1: Attention Exploration	2
1.1	1. Copying in Attention	2
1.2	2. An Average of Two	2
1.3	3. Drawbacks of Single-Headed Attention	4
1.4	4. Benefits of Multi-Headed Attention	5
2	Q2: In-Context Learning	6
2.1	1. Random Demonstrations	6
2.2	2. Demonstration Retrieval	6
2.3	3. Zero-Shot Learning	6
3	Q3: Textual Search	7
3.1	Sparse vs. Dense Search Results	7
3.2	When Sparse vs. Dense Search is Preferable	8
4	Q4: AI Disclosure & Reflection	8
4.1	AI Models Used	8
4.2	Usage Purpose	8
4.3	Reflection	8
5	Q5: Pretrained Transformers (Bonus)	9
5.1	Dev Accuracy Without Pretraining	9
5.2	London Baseline Accuracy	9
5.3	Dev Accuracy With Pretraining	9
5.4	Explanation of Performance Gain	9

1. Q1: Attention Exploration

1.1 1. Copying in Attention

(a) Categorical Interpretation of α

α can be seen as a categorical probability distribution over the value vectors $v_1, \dots, v_n \in \mathbb{R}^d$. It is easy to see that for all $i \in [n]$,

$$0 \leq \alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} \leq 1,$$

and

$$\sum_{i=1}^n \alpha_i = \sum_{i=1}^n \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} = \frac{\sum_{i=1}^n \exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} = 1.$$

(b) Conditions for Concentrated Attention

α becomes almost a one-hot vector (concentrated on some α_i) when one key k_i matches the query much more than the other keys, i.e.,

$$\forall j \in [n], j \neq i : \quad k_i^\top q \gg k_j^\top q.$$

(c) Output Vector c Under These Conditions

In this case, $\alpha_i \approx 1$ and $\alpha_j \approx 0$ for all $j \in [n], j \neq i$, so

$$c = \sum_{j=1}^n v_j \alpha_j \approx 1 \cdot v_i + \sum_{j \neq i} 0 \cdot v_j \approx v_i.$$

(d) Intuitive Explanation

If a key k_i matches the query q more than the other keys, the attention output c is almost a “copy” of the vector v_i .

1.2 2. An Average of Two

(a) Extracting v_a from $s = v_a + v_b$

We know that the orthonormal basis $\{a_1, \dots, a_m\}$ satisfies $\|a_i\| = 1$ and $a_i^\top a_j = 0$ for $i \neq j$. Hence $a_i^\top a_i = 1$. In addition, for any $j \in [p]$ we have $a_i^\top b_j = 0$.

As explained, v_a and v_b lie in the linear spaces $A = \text{span}\{a_i \mid i \in [m]\}$ and $B = \text{span}\{b_j \mid j \in [p]\}$, respectively. This means there exist scalars $\alpha_1, \dots, \alpha_m, \beta_1, \dots, \beta_p \in \mathbb{R}$ such that

$$v_a = \sum_{i=1}^m \alpha_i a_i, \quad v_b = \sum_{j=1}^p \beta_j b_j.$$

Define $A_i = a_i a_i^\top \in \mathbb{R}^{d \times d}$. Then

$$A_i a_i = a_i a_i^\top a_i = a_i \cdot 1 = a_i, \quad A_i b_j = a_i a_i^\top b_j = a_i \cdot 0 = 0,$$

and also

$$A_i a_j = a_i a_i^\top a_j = a_i \cdot 0 = 0 \quad (j \neq i).$$

With this definition, set

$$M = \sum_{k=1}^m A_k.$$

Then, letting $s = v_a + v_b$, we get

$$Ms = M(v_a + v_b) = Mv_a + Mv_b.$$

First,

$$\begin{aligned} Mv_a &= M \sum_{i=1}^m \alpha_i a_i = \sum_{i=1}^m \alpha_i M a_i = \sum_{i=1}^m \alpha_i \left(\sum_{k=1}^m A_k a_i \right) \\ &= \sum_{i=1}^m \alpha_i \left(\sum_{k=1}^m \begin{cases} a_i, & k = i, \\ 0, & k \neq i, \end{cases} \right) = \sum_{i=1}^m \alpha_i a_i = v_a. \end{aligned}$$

Second,

$$\begin{aligned} Mv_b &= M \sum_{j=1}^p \beta_j b_j = \sum_{j=1}^p \beta_j M b_j = \sum_{j=1}^p \beta_j \left(\sum_{k=1}^m A_k b_j \right) \\ &= \sum_{j=1}^p \beta_j \left(\sum_{k=1}^m 0 \right) = \sum_{j=1}^p \beta_j \cdot 0 = 0. \end{aligned}$$

Therefore,

$$Mv_a + Mv_b = v_a + 0 = v_a.$$

(b) Query Vector for Averaging Two Values

For $q = t(k_1 + k_2)$, for some large t , we get:

$$k_1^\top q = t(k_1^\top k_1 + k_1^\top k_2) = t(1 + 0) = t, \quad k_2^\top q = t(k_2^\top k_1 + k_2^\top k_2) = t(0 + 1) = t,$$

and for all $i \in [n] \setminus \{1, 2\}$,

$$k_i^\top q = t(k_i^\top k_1 + k_i^\top k_2) = t(0 + 0) = 0.$$

Thus, for α_1 we have

$$\alpha_1 = \frac{\exp(k_1^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} = \frac{\exp(t)}{2 \exp(t) + \sum_{j \in [n] \setminus \{1, 2\}} \exp(k_j^\top q)} \approx \frac{1}{2},$$

and similarly $\alpha_2 \approx \frac{1}{2}$.

For $i \in [n] \setminus \{1, 2\}$ we get

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} = \frac{\exp(0)}{2 \exp(t) + \sum_{j \in [n] \setminus \{1, 2\}} \exp(k_j^\top q)} = \frac{1}{2 \exp(t) + \sum_{j \in [n] \setminus \{1, 2\}} \exp(k_j^\top q)} \approx 0.$$

So finally,

$$c = \sum_{j=1}^n v_j \alpha_j \approx \frac{1}{2} v_a + \frac{1}{2} v_b.$$

1.3 3. Drawbacks of Single-Headed Attention

(a) Query Design with Small Isotropic Noise

Because the key vectors are sampled randomly $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, and the covariance matrices are defined as $\Sigma_i = \alpha I$ for vanishingly small α , we can effectively say that $k_i^\top q \approx \mu_i^\top q$.

So under the assumption that $\mu_i^\top \mu_j = 0$ for $i \neq j$ and $\|\mu_i\| = 1$, if we define $q = t(\mu_1 + \mu_2)$ for some large t , we get the same result as in the previous section. Namely,

$$k_1^\top q \approx \mu_1^\top q = t(\mu_1^\top \mu_1 + \mu_1^\top \mu_2) = t(1 + 0) = t,$$

$$k_2^\top q \approx \mu_2^\top q = t(\mu_2^\top \mu_1 + \mu_2^\top \mu_2) = t(0 + 1) = t,$$

and for all $i \in [n] \setminus \{1, 2\}$,

$$k_i^\top q \approx \mu_i^\top q = t(\mu_i^\top \mu_1 + \mu_i^\top \mu_2) = t(0 + 0) = 0.$$

Thus, for α_1 we have

$$\alpha_1 = \frac{\exp(k_1^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} \approx \frac{\exp(t)}{2 \exp(t) + \sum_{j \in [n] \setminus \{1, 2\}} \exp(k_j^\top q)} \approx \frac{1}{2},$$

and similarly $\alpha_2 \approx \frac{1}{2}$.

For $i \in [n] \setminus \{1, 2\}$ we get

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)} \approx \frac{\exp(0)}{2 \exp(t) + \sum_{j \in [n] \setminus \{1, 2\}} \exp(k_j^\top q)} = \frac{1}{2 \exp(t) + \sum_{j \in [n] \setminus \{1, 2\}} \exp(k_j^\top q)} \approx 0.$$

So finally we get:

$$c = \sum_{j=1}^n v_j \alpha_j \approx \frac{1}{2} v_a + \frac{1}{2} v_b.$$

(b) Effect of Variance in Key Magnitude

When repeatedly sampling the keys and using the vector q from the previous question, the vector c will vary significantly across different samples.

The vector c depends on the weights α_i , which depend on the attention scores $k_i^\top q$. This dot product is sensitive to the magnitude of each vector, due to the identity

$$k_i^\top q = \|k_i\| \|q\| \cos \theta,$$

where θ is the angle between the vectors.

Since k_a is roughly in the same direction as μ_a , the angle does not significantly affect $k_i^\top q$, but magnitude variations cause the attention scores to vary widely across different samples.

If $\|k_a\|$ is very large, its dot product with q will be larger than the others, causing the softmax to place most of the attention mass on it, so c will be more influenced by v_a .

If $\|k_a\|$ is very small, its dot product with q will be smaller than the others, causing the softmax to distribute the attention among the other keys, so c will resemble the form from the previous question.

1.4 4. Benefits of Multi-Headed Attention

(a) Designing q_1 and q_2

Because the key vectors are sampled randomly $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, and the covariance matrices are defined as $\Sigma_i = \alpha I$ for vanishingly small α , we can effectively say that $k_i^\top q \approx \mu_i^\top q$.

Let us define:

$$c = \frac{1}{2}(c_1 + c_2), \quad c_1 = \sum_{i=1}^n v_i \alpha_i^{(1)}, \quad c_2 = \sum_{i=1}^n v_i \alpha_i^{(2)}.$$

Under the assumption that $\mu_i^\top \mu_j = 0$ for $i \neq j$ and $\|\mu_i\| = 1$, if we define $q_1 = t\mu_1$ and $q_2 = t\mu_2$ for some large t , we obtain the desired result.

For q_1 we have

$$k_1^\top q_1 \approx \mu_1^\top q_1 = t(\mu_1^\top \mu_1) = t \cdot 1 = t,$$

and for all $i \in [n] \setminus \{1\}$,

$$k_i^\top q_1 \approx \mu_i^\top q_1 = t(\mu_i^\top \mu_1) = t \cdot 0 = 0.$$

The weights are therefore

$$\alpha_1^{(1)} = \frac{\exp(k_1^\top q_1)}{\sum_{j=1}^n \exp(k_j^\top q_1)} \approx \frac{\exp(t)}{\exp(t) + \sum_{j \in [n] \setminus \{1\}} \exp(k_j^\top q_1)} \approx 1,$$

and for $i \in [n] \setminus \{1\}$,

$$\alpha_i^{(1)} = \frac{\exp(k_i^\top q_1)}{\sum_{j=1}^n \exp(k_j^\top q_1)} \approx \frac{\exp(0)}{\exp(t) + \sum_{j \in [n] \setminus \{1\}} \exp(k_j^\top q_1)} = \frac{1}{\exp(t) + \sum_{j \in [n] \setminus \{1\}} \exp(k_j^\top q_1)} \approx 0.$$

For q_2 the argument is the same, yielding $\alpha_2^{(2)} \approx 1$ and $\alpha_i^{(2)} \approx 0$ for all $i \in [n] \setminus \{2\}$.

Summarizing, we get

$$\begin{aligned} c_1 &= \sum_{i=1}^n v_i \alpha_i^{(1)} \approx 1 \cdot v_1 + 0 \cdot v_2 + \sum_{i=3}^n 0 \cdot v_i = v_1, \\ c_2 &= \sum_{i=1}^n v_i \alpha_i^{(2)} \approx 0 \cdot v_1 + 1 \cdot v_2 + \sum_{i=3}^n 0 \cdot v_i = v_2, \end{aligned}$$

and therefore

$$c = \frac{1}{2}(c_1 + c_2) = \frac{1}{2}(v_1 + v_2).$$

(b) Expected Output Variance

When repeatedly sampling the keys and using the vector q from the previous question, the vector c will be approximately equivalent across different samples.

The difference now is that the attention output c is calculated as the average of the attention outputs of the two heads, c_1 and c_2 .

Since each attention head output c_1, c_2 is focused on a specific value vector v_a and v_b , even a drastic change in the magnitude of k_a will not change the softmax result within each head. This means that the attention mass in each head will not shift to another vector, and their outputs will remain stable.

2. Q2: In-Context Learning

2.1 1. Random Demonstrations

(a) Beam Search Results

We randomly sampled six sets of in-context demonstrations from the 500 available TriviaQA examples, with sizes ranging from 3 to 8. For each set, we constructed a prompt in the required Question–Answer format and evaluated GPT2-medium on the TriviaQA validation set using beam search decoding. The results were reported as the average correctness over the validation set using a pandas DataFrame and were lower than what we’ve expected.

(b) Sampling Decoding Comparison

We repeated the same evaluation using sampling decoding with temperature 0.7. Compared to beam search, sampling produced more diverse but less stable outputs, which led to lower accuracy. The randomness in sampling often caused the model to generate less precise factual answers.

2.2 2. Demonstration Retrieval

(a) Retrieved Demonstrations Results

We first encoded all questions from the optional in-context demonstrations using BERT. For each validation question, we retrieved the 8 most similar demonstrations based on the dot product between their embedding vectors. Each validation example was evaluated with its own retrieved prompt, and results were reported for both beam search and sampling decoding.

(b) Qualitative Explanation

The retrieved demonstrations are more semantically similar to the validation question and often include related entities or facts. This gives GPT-2 more relevant context, which helps it generate better answers compared to using random demonstrations.

2.3 3. Zero-Shot Learning

(a) Zero-Shot Evaluation Results

In the zero-shot setting, GPT2-medium was evaluated on the TriviaQA validation set without providing any in-context demonstrations. The model achieved low accuracy, which was reported using the same evaluation method as in previous sections.

(b) Explanation for Low Performance

Without demonstrations, GPT-2 has no guidance on the expected answer format or relevant information. As a result, it often generates incomplete or incorrect answers, leading to low performance.

(c) LAMA Evaluation Results

To address the difficulty of zero-shot question answering, we evaluated GPT-2 on the LAMA sentence-completion benchmark. In this setting, the model completes masked factual sentences, and performance on the LAMA validation set was reported.

(d) Analysis of Model Generations

LAMA performs only slightly better than TriviaQA in the zero-shot setting and performs worse when using sampling. Many correct predictions seem to come from common language patterns rather than true factual knowledge.

3. Q3: Textual Search

3.1 Sparse vs. Dense Search Results

(a) Query: What is the capital of New Zealand?

Sparse Search (TF-IDF):

1. (0.9084) “New Zealand”
2. (0.7286) “New Zealand’s capital city is Wellington, and its most populous city is Auckland”
3. (0.4824) “The South Island is the largest landmass of New Zealand”

Dense Search (Inner Product):

1. (0.7761) “New Zealand’s capital city is Wellington, and its most populous city is Auckland”
2. (0.6516) “New Zealand is an island country in the southwestern Pacific Ocean”
3. (0.6515) “New Zealand country profile from BBC News
Te Ara: The Encyclopedia of New Zealand
New Zealand”

(b) Query: What currency is used in New Zealand?

Sparse Search (TF-IDF):

1. (0.9084) “New Zealand”
2. (0.8258) “23% in 2015”
3. (0.4664) “The word today is increasingly used to refer to all non-Polynesian New Zealanders”

Dense Search (Inner Product):

1. (0.7660) “The currency is the New Zealand dollar, informally known as the “Kiwi dollar”; it also circulates in the Cook Islands (see Cook Islands dollar), Niue, Tokelau, and the Pitcairn Islands”
2. (0.5950) “New Zealand’s main trading partners, , are China (NZ\$27”
3. (0.5704) “6%) to New Zealand’s total GDP and supporting 7”

(c) Query: Should I visit New Zealand if I am interested in sightseeing?

Sparse Search (TF-IDF):

1. (0.9084) “New Zealand”
2. (0.8258) “23% in 2015”

3. (0.4433) “If no majority is formed, a minority government can be formed if support from other parties during confidence and supply votes is assured”

Dense Search (Inner Product):

1. (0.5303) “New Zealand is known for its extreme sports, adventure tourism and strong mountaineering tradition, as seen in the success of notable New Zealander Sir Edmund Hillary”
2. (0.5136) “New Zealand country profile from BBC News
Te Ara: The Encyclopedia of New Zealand
New Zealand”
3. (0.5127) “New Zealand art and craft has gradually achieved an international audience, with exhibitions in the Venice Biennale in 2001 and the “Paradise Now” exhibition in New York in 2004”

3.2 When Sparse vs. Dense Search is Preferable

Sparse search works better when looking for specific facts or definitions, where keyword overlap is likely to produce relevant results. In such cases, matching exact terms is usually sufficient to find the correct documents. On the other hand, dense search is more suitable for general or abstract queries that require a deeper level of understanding, since the embeddings capture semantic information beyond simple word overlap.

4. Q4: AI Disclosure & Reflection

4.1 AI Models Used

Chat-GPT 5.2

4.2 Usage Purpose

1. L^AT_EX formatting help
2. Help with brainstorming and concept understanding
3. Help understanding 're' python package

4.3 Reflection

The AI tools made working on Assignment 3 easier and more efficient, often helping us think through and refine our ideas along the way. One downside was that we sometimes needed to give more background than expected to get responses that were truly accurate and useful.

Working on the bonus question helped clarify the practical difference between training a model from scratch and leveraging pretraining on a large, knowledge-rich corpus. Implementing the full pipeline - from span corruption pretraining to finetuning and evaluation - made the role of pretraining in enabling generalization very concrete. The most challenging part was managing the longer training process, but seeing the significant performance improvement made the effort worthwhile.

5. Q5: Pretrained Transformers (Bonus)

5.1 Dev Accuracy Without Pretraining

The model accuracy on the dev set (trained from scratch, no pretraining) was **1.6%**.

5.2 London Baseline Accuracy

The “London” baseline (predicting *London* for every example in the dev set) achieved **5.00%** accuracy.

The model without pretraining performs worse than simply predicting “London” for every example. This suggests that the model cannot generalize birthplace predictions from the limited name–birthplace training set alone, and lacks the external world knowledge needed for many people in the dev set.

5.3 Dev Accuracy With Pretraining

After pretraining the model on `wiki.txt` using the span corruption objective and then finetuning on the name–birthplace dataset, the model achieved a dev accuracy of **24.8%**.

5.4 Explanation of Performance Gain

The pretrained model achieves substantially higher accuracy because it is exposed to large amounts of world knowledge during pretraining. The span corruption objective requires the model to reconstruct masked spans from Wikipedia text, which encourages it to encode factual associations, including relationships between notable people and geographic locations.

In contrast, the non-pretrained model is trained only on a small supervised dataset of name–birthplace pairs and therefore lacks prior knowledge about unseen individuals in the dev set. As a result, it cannot generalize beyond memorizing frequent or trivial patterns. Finetuning the pretrained model teaches it the specific question–answering format, while the factual knowledge required to answer the questions is largely acquired during pretraining.